

sxchan: An R Package for Channel Cross-Section Analysis

2025-08-19

Summary

sxchan is an R package that helps researchers analyze river channels by creating and manipulating cross-sectional data. River channels can be described in two ways: as horizontal lines across the channel (planimetric cross-sections) and as elevation profiles along those lines (profile cross-sections). The package allows users to generate these cross-sections from various data sources including river bank outlines, digital elevation models, or simple width specifications. Once created, users can erode the banks to create newly eroded channels. Channel properties can be calculated, like width and depth, centerlines generated and banklines traced. The package integrates with existing R spatial analysis tools, making it easy to incorporate into larger research workflows for studying river processes, erosion, and channel evolution in a programmatic way.

Statement of Need

River channel analysis requires handling both planimetric cross-sections (horizontal line segments extending bank-to-bank) and profile cross-sections (elevation data along those segments). Current approaches often require manual GIS workflows or separate tools that don't integrate well with the R ecosystem or automation. Existing R packages for spatial analysis (e.g., **sf** (Pebesma 2018), **terra** (Hijmans 2023)) provide excellent tools for individual operations but lack specialized functionality for channel cross-section analysis.

The advent of remote sensing technologies, particularly airborne LiDAR and photogrammetry, has provided the ability to capture high-resolution topographic data over large spatial extents (Milan et al. 2011; Westaway, Lane, and Hicks 2003). These technologies enable the generation of digital elevation models (DEMs) that can be used to extract detailed channel and floodplain geometry information. However, the challenge lies in efficiently processing these data to extract meaningful cross-sectional information and integrate it with other spatial datasets.

Current approaches to channel analysis rely on manual GIS workflows using software such as ArcGIS or QGIS, where users must manually digitize cross-sections, extract elevation profiles, and perform geometric calculations through point-and-click interfaces. These workflows are time-consuming, error-prone, and difficult to reproduce. While R packages like **sf** and **terra** provide excellent tools for spatial data manipulation and raster processing, they lack specialized functionality for channel cross-section analysis. Researchers must cobble together multiple packages and custom scripts to achieve what should be a unified workflow: generating cross-sections from banklines, sampling DEMs for elevation data, and analyzing channel geometry properties.

This gap is particularly evident in the context of fluvial geomorphology research, where researchers need to analyze channel geometry changes over time, assess erosion and deposition patterns, and model channel evolution processes (Church 2006; Eaton, Church, and Millar 2010). The **sxchan** package addresses this gap by providing a unified framework for channel cross-section analysis that integrates seamlessly with the existing R spatial analysis ecosystem.

Key Features

The package provides a unified data structure for channel objects that integrates planimetric and profile cross-sections. Plan-view cross-sections extend from bank to bank, while profile cross-sections are elevation

profiles along those lines. To enable informed erosion modeling into the floodplain, profile cross-sections can extend beyond the banks into the surrounding topography.

Data Structures

The package deals with two main object types:

- **Channels:** The primary objects, comprised of cross-sections
- **Profile cross-sections:** Specialized geometry objects requiring custom handling

Plan-view cross-sections are represented as simple line segments (using `sf` geometries), avoiding unnecessary complexity and keeping the package lightweight.

Channel Objects

Channels are implemented as data frames (or tibbles (Müller and Wickham 2020) when available), with plan and profile cross-sections stored as columns. Additional columns can represent channel properties at each cross-section, following the design philosophy of the `sf` package where spatial objects are just enhanced data frames.

Profile Cross-Sections

Profile cross-sections are specified as coordinate matrices with distance along the profile in the first column and elevation in the second. They encode bank locations and thalweg points (potentially multiple thalwegs for complex channels).

Design Philosophy

The package prioritizes human readability and consistency. Functions are prefixed with `xt_` (for “cross-section type”) to avoid conflicts and improve code clarity, following the pattern established by the `sf` package. The `units` package ensures consistent dimensional analysis throughout. The gradient calculation function draws inspiration from the `slider` package’s sliding window approach for time series analysis (Vaughan 2024).

Function Factory Architecture

A core architectural decision is the consistent use of function factories for configurable operations. This pattern elegantly solves the tension between API simplicity and extensibility that plagues many scientific software packages. Rather than creating separate functions for each algorithm variant (e.g., `xt_widen_linear()`, `xt_widen_spline()`, `xt_widen_custom()`), the package uses factory functions that return specialized algorithms while maintaining a unified interface.

This design provides several key advantages:

Consistent User Experience: All configurable operations follow the same pattern. Users learn once how to use `method = "simple"` or `method = algorithm(parameters)`, and this knowledge transfers across side functions, tracers, and dredgers.

Parameter Capture: Factory functions elegantly capture configuration parameters in closures, avoiding the need for complex parameter passing or global state.

Extensibility Without API Bloat: New algorithms can be added without changing the core function signatures. Users can create custom implementations that integrate seamlessly with the existing API.

R Ecosystem Integration: This pattern is widely used in the R ecosystem (e.g., `ggplot2` geoms, `dplyr` verbs, `scales` transforms), making the package feel familiar to R users.

The package implements five function factory systems:

Specification Factories create functions that define parameters and contexts for operations:

- **Side Factories** create functions that specify erosion allocation during channel widening, allowing users to define how erosion is distributed between left and right banks. Built-in factories handle common cases (e.g., “left”, “right”, “both”), enabling sophisticated erosion patterns based on bank properties, channel geometry, or environmental factors.
- **Elevation Factories** create functions for flexible elevation specification across all operations, enabling users to specify elevation sources from thalweg depths to water surface columns to custom calculations.

Operator Factories create functions that perform actual operations on channels:

- **Dredger Factories** create functions for channel modification operations, supporting different algorithms for creating new channel geometries. This includes rectangular dredging for simple cases and spline-based dredging for smooth curve generation based on surrounding topography.
- **Tracer Factories** create functions for bankline generation from planimetric cross-sections, enabling different algorithms for connecting bank points into continuous banklines. Users can choose from simple linear connections for straight channels to sophisticated curve-fitting algorithms for meandering rivers.
- **Extender Factories** create functions for extending the topographic frame beyond channel banks, enabling users to add floodplain topography for channel widening simulations. Built-in factories support flat extensions and sloped extensions with configurable gradients.

The factory functions create “first-class citizens” - objects that can be passed as arguments, returned from functions, and composed together. This enables sophisticated parameter capture, such as embedding elevation specifications within dredger functions (e.g., `dredger_rectangle(depth = 5, wse = elevation_column("water_surface"))`), allowing each algorithm to capture its specific requirements while maintaining a consistent interface.

This architectural approach transforms what could be a collection of disparate functions into a cohesive, extensible system that grows with user needs rather than requiring API changes.

Channel Frame Extension

Channel frame extension adds topographic context beyond the banks to enable realistic channel widening simulations. The extender system supports flat extensions for simple floodplains and sloped extensions for realistic terrain gradients. Users can specify different extension patterns on left and right sides to model asymmetric topography, such as hills on one side and floodplains on the other.

Channel Operations

Channel Widening

Channel widening simulates bank erosion processes. For plan-view cross-sections, widening simply extends the line segments. For profile cross-sections, the approach preserves the existing profile while widening the channel from the thalweg outward. This means the channel bottom (thalweg) becomes wider, with the river section between the bank and thalweg shifting laterally, and corresponding beyond-bank topography being removed.

Channel Dredging

Channel dredging modifies profile cross-sections to create new channel geometries. The dredger system supports various algorithms, from simple rectangular dredging to sophisticated spline-based methods that generate smooth curves based on surrounding topography. Users can specify water surface elevations or rely on automatic detection from bank heights.

Derived Geometries

The package provides functions to generate derived geometries from channel objects. Centerlines are created by connecting the midpoints of planimetric cross-sections, while banklines are generated using the tracer

system to connect bank points into continuous banklines.

Example Usage

```
library(sxchan)

# Load data
banks <- sf::st_read("river_banks.gpkg")
dem <- terra::rast("elevation.tif")

# Generate channel planimetric cross-sections from demo bankline data
channel <- xt_channel_from_banks(demo_bankline, n = 100)
plot(demo_bankline, col = "lightblue")
plot(channel, view = "plan", add = TRUE)

# Sample DEM for profile cross-sections
plot(demo_dem)
channel <- xt_sample_dem(dem, extent_multiplier = 2, sample_n = 100)

# Dredge the channel using different methods
channel_rect <- xt_dredge(channel, method = dredger_rectangle(depth = 5))
channel_smooth <- xt_dredge(channel, method = dredger_spline(alpha = 0.3))
plot(channel_rect, view = "3d")

# Widen the channel with custom erosion allocation
channel_widened <- xt_widen(channel, width = 10, side = side_left(prop = 0.75))
plot(channel_widened, view = "3d")

# Generate derived geometries
centerline <- xt_centerline(channel)
banks <- xt_trace_banks(channel, tracer = "linear")
banks_smooth <- xt_trace_banks(channel, tracer = tracer_spline(degree = 3))

# Extend topographic frame for widening simulations
channel_extended <- xt_extend_frame(channel, extender = extender_flat(extent = 20))
channel_asymmetric <- xt_extend_frame_left(channel, extender = extender_slope(extent = 30, slope = 0.05))

# Analyze results
heights <- xt_height(channel, .f = min)
widths <- xt_width(channel)
```

Acknowledgments

The authors thank BGC Engineering Inc. for funding this work. We also thank Brett Eaton and Sarah Davidson for guiding the package utility, and Heba Abdelmoaty for contributing to algorithm design and the developers of the `sf`, `terra`, `units`, and `scales` packages for providing the foundational tools and design inspiration that make `sxchan` possible.

References

Church, Michael. 2006. "Bed Material Transport and the Morphology of Alluvial River Channels." *Annual Review of Earth and Planetary Sciences* 34: 325–54.

- Eaton, Brett C, Michael Church, and Robert G Millar. 2010. “Predicting Downstream Hydraulic Geometry: A Test of Rational Regime Theory.” *Journal of Geophysical Research: Earth Surface* 115 (F4).
- Hijmans, Robert J. 2023. “Terra: Spatial Data Analysis.” *R Package Version 1* (7): 71.
- Milan, David J, George L Heritage, Andrew RG Large, and Neil S Entwistle. 2011. “Terrestrial Laser Scan-Derived Topographic and Roughness Data for Hydraulic Modelling of Gravel-Bed Rivers.” *Earth Surface Processes and Landforms* 36 (10): 1334–48.
- Müller, Kirill, and Hadley Wickham. 2020. “Tibble: Simple Data Frames.” *R Package Version 3* (1): 1.
- Pebesma, Edzer. 2018. “Simple Features for r: Standardized Support for Spatial Vector Data.” *The R Journal* 10 (1): 439–46.
- Vaughan, Davis. 2024. *Slider: Sliding Window Functions*. <https://CRAN.R-project.org/package/slider>.
- Westaway, Richard M, Stuart N Lane, and D Murray Hicks. 2003. “Remote Sensing of Clear-Water, Shallow, Gravel-Bed Rivers Using Digital Photogrammetry.” *Photogrammetric Engineering & Remote Sensing* 69 (11): 1273–84.